House Prices

Advanced Regression on the Ames, Iowa Housing Dataset

Stacked Ridge Meta-Learner over 7 Base Models — EDA, Cleaning, Feature Engineering, Tuning & Explainability

0.10740	+5.4%	80 → 251
Stacker 5-fold OOF RMSLE	vs Ridge baseline	raw → engineered features

1. Business Problem

A real-estate company wants to estimate the selling price of a house **before** it is listed. A reliable pre-listing valuation model lets the business set competitive prices, sell faster, and avoid the cost of over- or under-pricing inventory.

Business benefits

- **Better pricing strategy** — data-driven list prices instead of guesswork.
- **Faster sales** — correctly priced homes spend less time on market.
- **Reduced over/under-pricing risk** — fewer mark-downs and missed margin.
- **Acquisition screening** — flag homes priced below model value.
- **Renovation ROI** — quantify which upgrades move price most.

Target variable: SalePrice, modeled on the **log1p** scale. The competition is scored on RMSLE (Root Mean Squared Log Error), which is mathematically equivalent to RMSE on log(SalePrice) — so predicting in log space aligns the loss with the metric.

2. Dataset and Target

The Kaggle House Prices dataset describes residential homes in **Ames, Iowa** using **80 explanatory features** (1,460 train / 1,459 test rows). After removing two documented outliers, **1,458 rows** are used for training. Features span every dimension a buyer cares about:

Category	Example features
Area	LotArea, GrLivArea, TotalBsmtSF, 1stFlrSF
Quality	OverallQual, OverallCond, KitchenQual
Location	Neighborhood, Condition1
Age	YearBuilt, YearRemodAdd, YrSold
Garage	GarageArea, GarageCars, GarageType
Basement	TotalBsmtSF, BsmtFinSF1, BsmtQual

2.1 Why log-transform the target?

Raw SalePrice has a strong right skew (**1.88**) — a long tail of luxury homes pulls the mean above the median. After **log1p**, skewness drops to **0.12** and the QQ plot is nearly linear, satisfying linear-model assumptions and weighting proportional errors equally across price ranges.

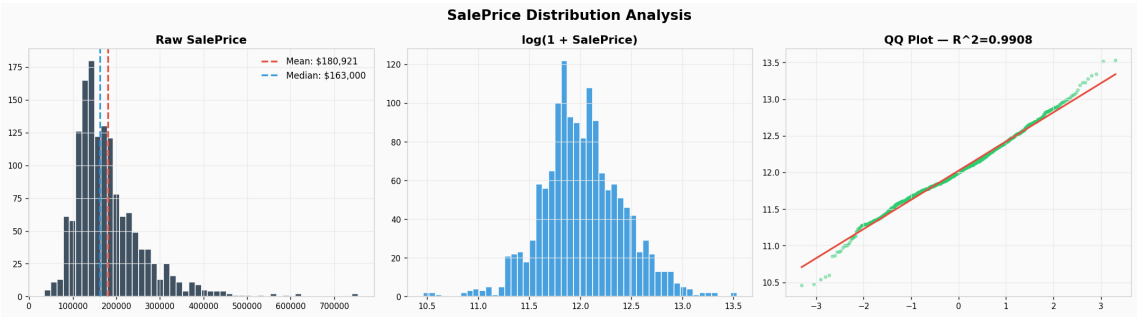

Fig 1. Raw SalePrice (left), log1p-transformed (middle), and QQ plot vs Normal (right).

3. Exploratory Data Analysis

3.1 Missing values: most are not actually missing

19 columns have missing values, but the data dictionary reveals that most NaN entries encode the **absence** of a feature, not unknown data. PoolQC is 99.5% NaN simply because almost no home has a pool. Filling these with a median would invent pools where there are none; the pipeline fills them with the literal string "None" so encoding treats them as a category.

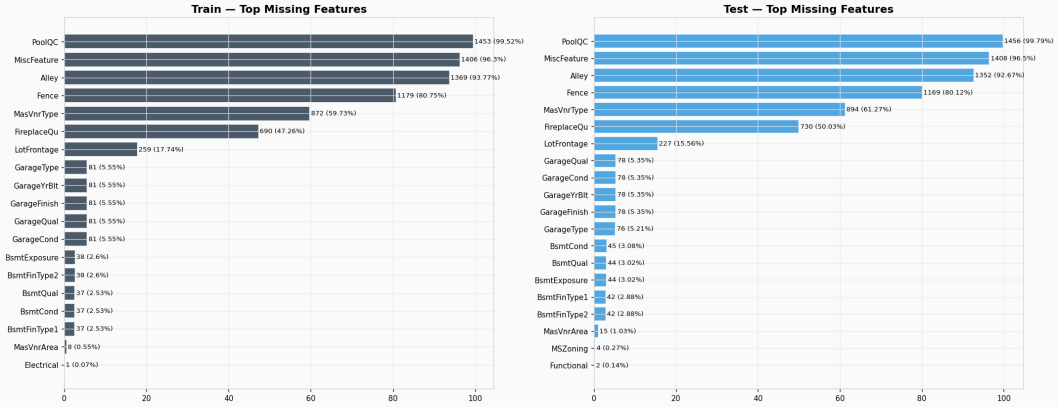

Fig 2. Top missing features in train (left) and test (right). Leading entries are all absence-encodings.

3.2 Correlations with target

OverallQual correlates **0.79** with SalePrice — by far the strongest single predictor. Five features clear the 0.6 mark, covering the obvious drivers: overall quality, living area, garage capacity, basement size and first floor size. GarageCars and GarageArea are nearly redundant.

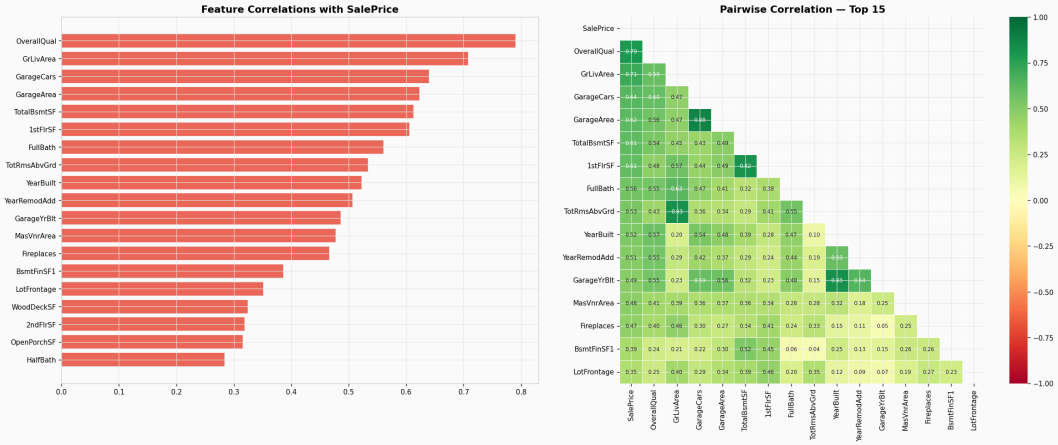

Fig 3. Top numeric features by absolute correlation (left); pairwise correlation heatmap (right).

3.3 Key price drivers

Plotting the six strongest predictors against SalePrice confirms quality and size dominate. Two GrLivArea points beyond 4,000 sqft sit far below trend — documented partial sales removed before training.

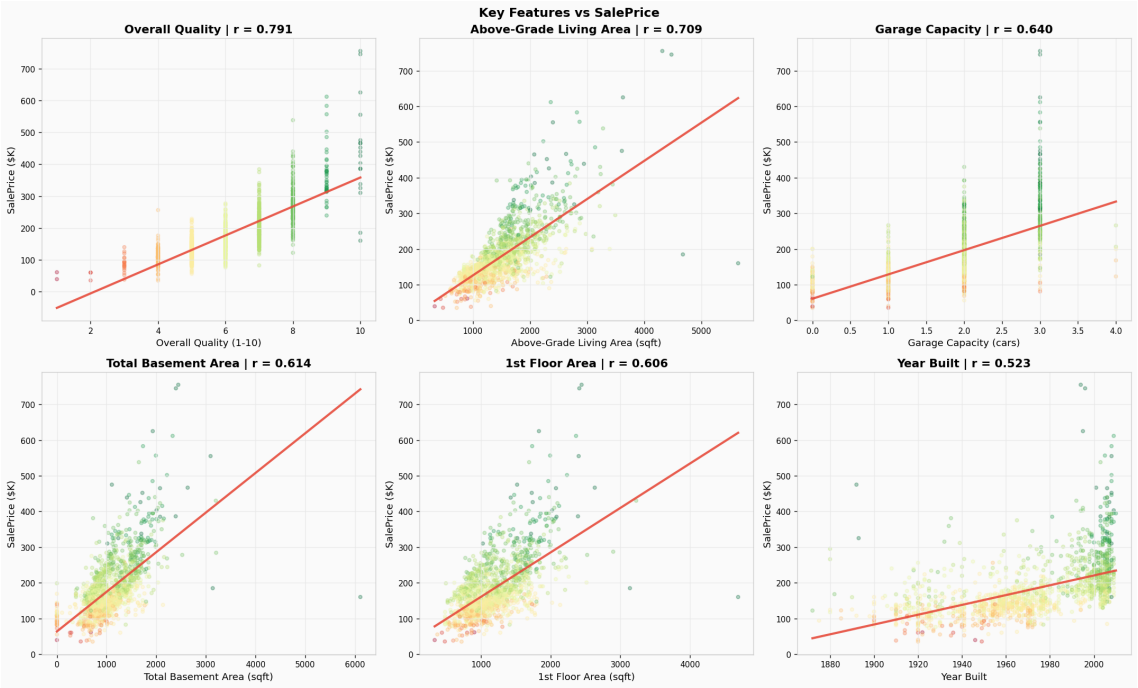

Fig 4. Six strongest predictors plotted against SalePrice.

3.4 Neighborhood premium

Median price varies **3.6×** between the cheapest (MeadowV) and most expensive (NridgHt) neighborhoods. This single categorical encodes a large fraction of total variance — exploited twice: as a target-encoded numeric feature and via KMeans clustering.

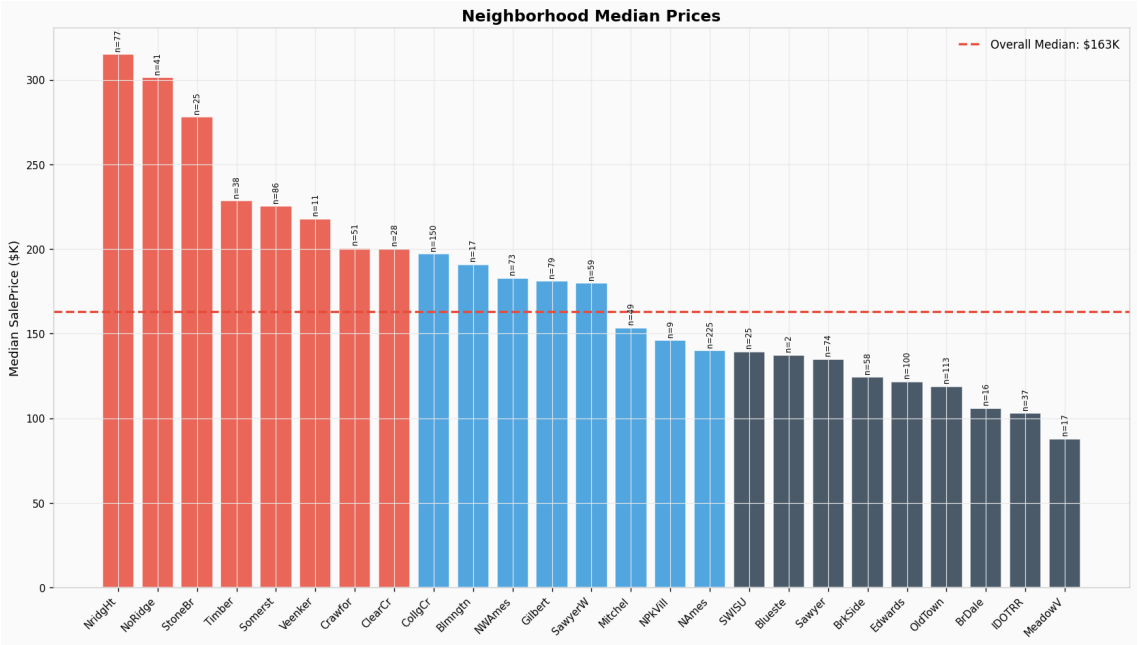

Fig 5. Neighborhood-level median sale prices, sorted descending (sample sizes shown per bar).

4. Data Cleaning & Missing Values

Missing values were imputed with **semantic, domain-aware** rules rather than blind defaults, and two documented outliers (homes > 4,000 sqft that sold for < $300K) were removed.

Strategy	Rationale
Fill with "None"	NaN encodes feature absence (PoolQC, Alley, Fence, FireplaceQu, Garage*, Bsmt*, MasVr
Fill with 0	No feature -> zero size/count (GarageArea/Cars, TotalBsmtSF, BsmtFinSF1/2, MasVnrArea
Neighborhood median	LotFrontage — houses on the same street share a frontage profile
Mode	Single-NaN columns (MSZoning, Electrical, KitchenQual, Functional, Utilities)

Of the missing columns, only **LotFrontage** (17.7% NaN) is truly missing data; the rest are absence-encodings.

5. Feature Engineering

Buyers think holistically — total space, total bathrooms, recency. The pipeline adds **17 engineered features**, then applies log1p to 42 skewed numerics and one-hot encoding, expanding the matrix to **251 model-ready features**.

Feature	Definition
TotalSF	TotalBsmtSF + 1stFlrSF + 2ndFlrSF
TotalBathrooms	FullBath + 0.5*HalfBath + BsmtFullBath + 0.5*BsmtHalfBath
HouseAge / YearsSinceRemod	Temporal recency features
Has* flags	HasPool, HasGarage, Has2ndFloor, HasBsmt, HasFireplace
QualArea / QualTotalSF	OverallQual x area — quality-weighted size
NeighborhoodPrice	Target encoding (median price per nbhd, train-only)
NeighborhoodCluster	KMeans (k=5) on aggregated neighborhood stats

Leakage guard: target encoding and KMeans clustering are fit on **train data only**, then mapped onto test. Fitting on the train+test concatenation would leak the test distribution and give optimistic CV scores.

6. Modeling & Hyperparameter Tuning

Seven base models were validated with **5-fold KFold** (shuffle, seed 42) on the log target:

Model	Role
Ridge / Lasso / ElasticNet	Linear baselines (L2 / L1 / hybrid)
GBM (sklearn)	Tree boosting with Huber loss
XGBoost	Gradient boosting, strong on interactions
LightGBM	Fast histogram-based boosting
CatBoost	Native categorical handling

Tuning & stacking

- **XGBoost, LightGBM, CatBoost** tuned with **Optuna** (TPE sampler, 30–100 trials), optimizing 5-fold CV RMSLE.
- **Ridge meta-learner** (RidgeCV) fit on the 5-fold OOF predictions of all 7 base models — true stacking, not fixed-weight blending.
- The meta-learner can assign negative coefficients and selects regularization via cross-validated alpha search.

7. Results & Model Comparison

Cross-validation scorecard (30 Optuna trials per booster). Lower OOF RMSLE is better.

Model	OOF RMSLE	vs Ridge baseline
Stacker (Ridge meta-learner)	**0.10740**	**+5.4%**
Lasso	0.11179	+1.5%
ElasticNet	0.11182	+1.5%
CatBoost (tuned)	0.11318	+0.3%
Ridge	0.11351	baseline
GBM	0.11395	-0.4%
XGBoost (tuned)	0.11556	-1.8%
LightGBM (tuned)	0.11602	-2.2%

Winner: the Ridge stacker at OOF RMSLE **0.10740** — a **5.4% improvement** over the Ridge baseline. Notably, the best *single* model was **CatBoost (0.11318)**; even it lost to the stacker. Lasso and ElasticNet outscored all three tuned tree boosters — on a small, clean dataset, simple often wins.

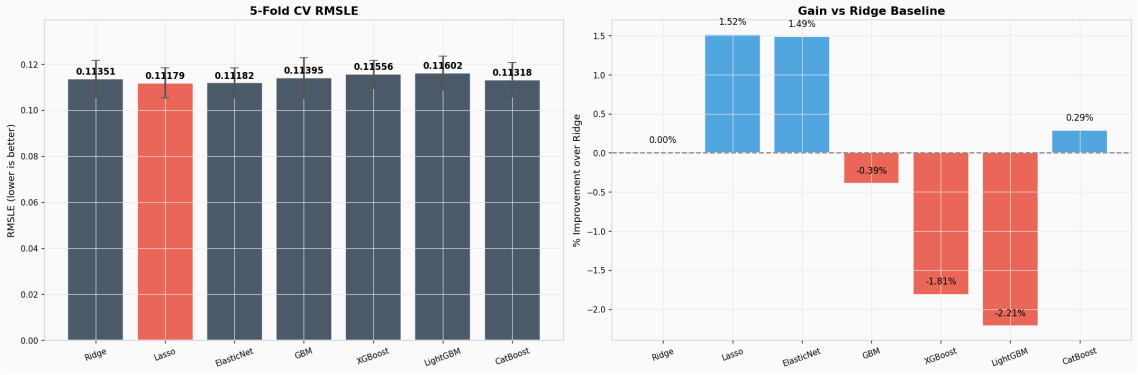

Fig 6. 5-fold CV RMSLE for all 7 base models with 1-sigma error bars (left) and percent improvement vs Ridge (right).

8. Stacker Diagnostics

Residuals are centered on zero with no strong heteroscedasticity. The predicted-vs-actual scatter hugs the y=x line tightly, with slight under-prediction at the upper extreme — expected for a model trained on a long-tailed target. Mean OOF RMSLE: **0.10740**.

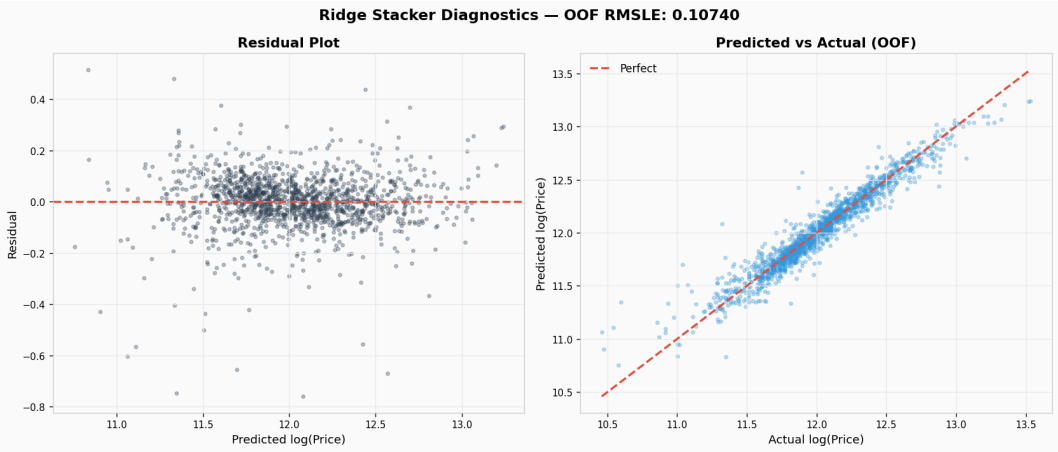

Fig 7. Stacker diagnostics — residual plot (left) and OOF predicted vs actual (right).

Feature importance

Engineered features dominate every tree model's rankings. **QualTotalSF**, **QualArea** and **NeighborhoodPrice** consistently appear in the top 5 — validation that the engineering work paid off.

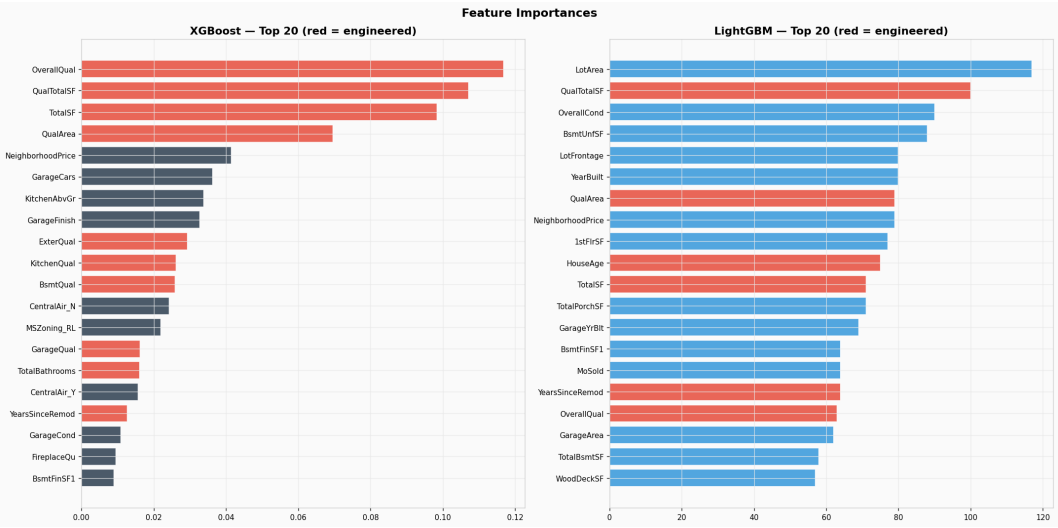

Fig 8. Top 20 features by importance for the tuned tree models (engineered features highlighted).

9. Explainability (SHAP)

SHAP values decompose each prediction into per-feature contributions, accounting for interactions — a more faithful picture than split-count importances. **QualTotalSF** (quality × total area) dominates as the single most informative composite feature.

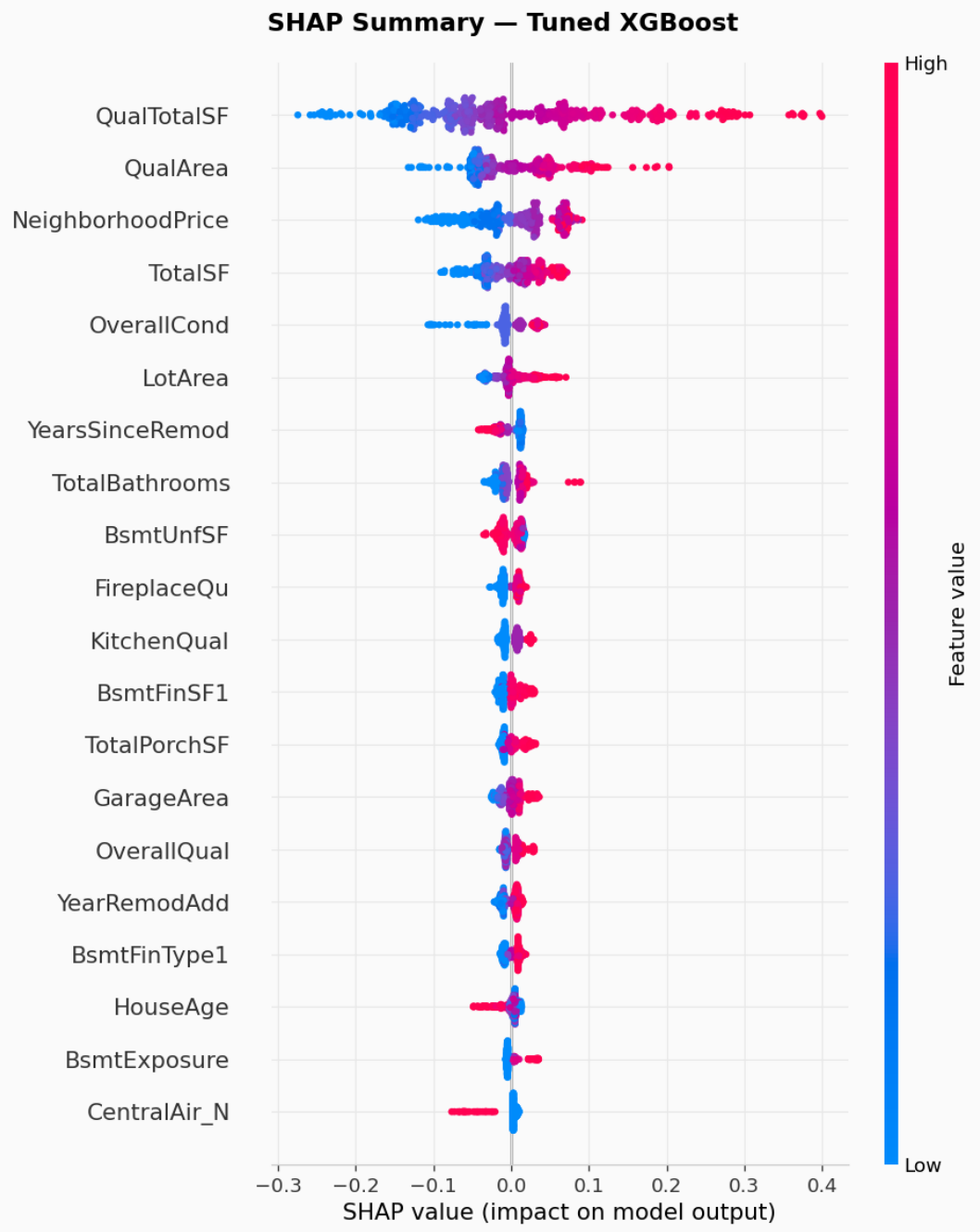

Fig 9. SHAP summary for the tuned XGBoost model. Color encodes feature value (red=high, blue=low); x-axis shows impact on predicted log price.

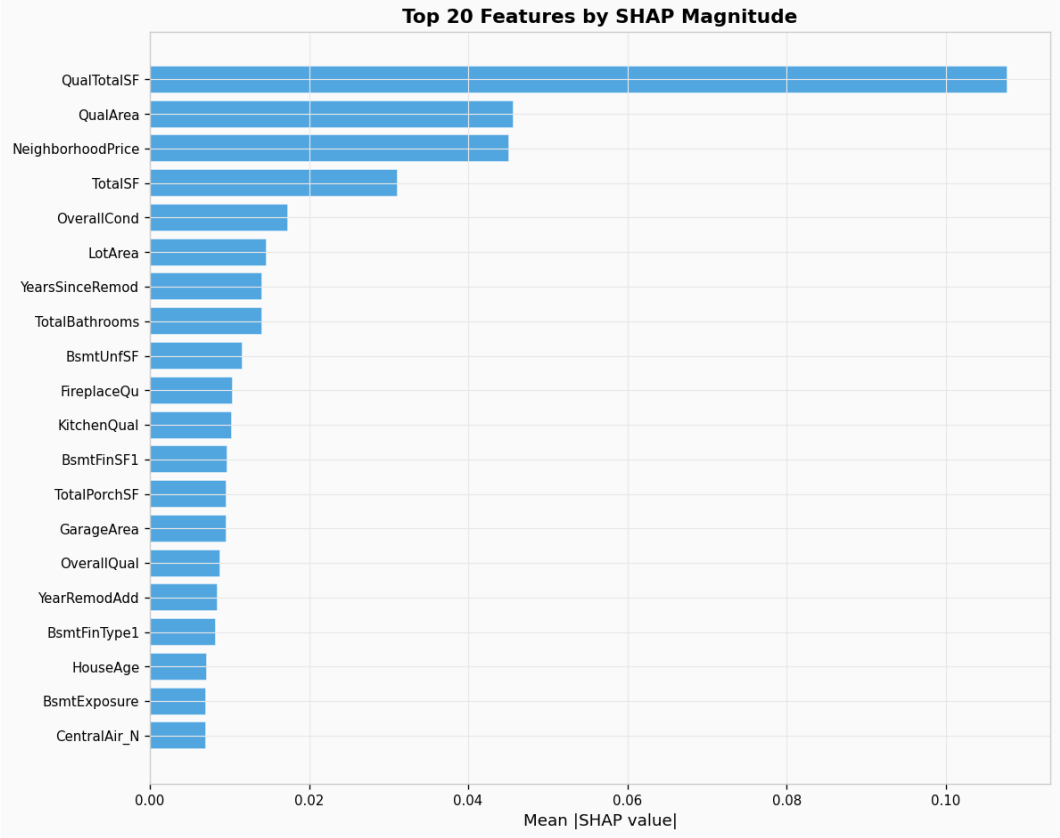

Fig 10. Top 20 features by mean |SHAP value|.

Prediction sanity check

Predicted test prices closely match the actual training-price distribution — no obvious extrapolation pathology.

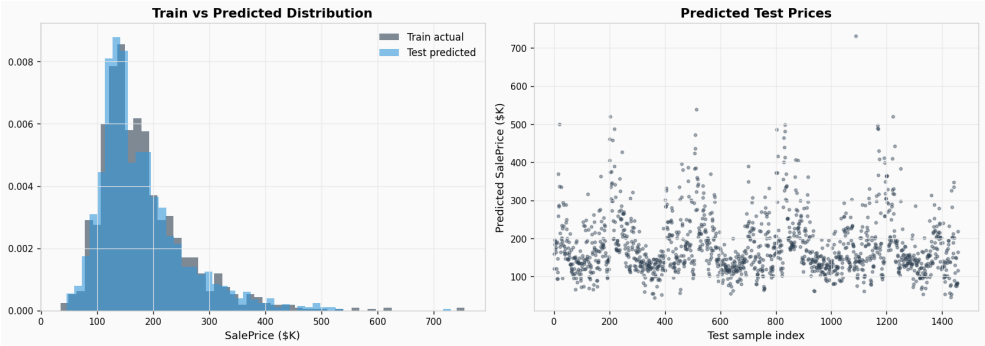

Fig 11. Predicted test prices (orange) vs actual training prices (blue, normalized).

10. Business Value & Future Work

The stacked model converts raw property attributes into a defensible, auditable price estimate the business can act on:

- **Pricing engine:** an instant pre-listing valuation, deployed via the Streamlit estimator over the serialized artifacts/model.pkl.
- **Faster turnover:** accurate prices reduce time-on-market and costly mark-downs.
- **Acquisition screening:** flag homes priced below model value as investment candidates.
- **Renovation ROI:** SHAP shows quality-weighted area moves price most — guiding where upgrade spend pays back.
- **Transparency:** SHAP explanations make every valuation auditable.

Key takeaways

- Log-transforming the target aligned the loss with RMSLE and tamed luxury outliers.
- Most NaN values encoded absence, not unknown data — semantic imputation mattered.
- Engineered features (QualTotalSF, QualArea) outranked every raw column.
- Stacking beat the best single model: CatBoost 0.113 lost to the Ridge stacker 0.107.

Future improvements

- **More Optuna trials** (--trials 100) typically pushes the stacker below 0.107.
- **External data:** school ratings, crime rate, distance to city center.
- **Deployment hardening:** wrap the saved model behind a monitored API.
- **BI dashboard** for neighborhood ranking and predicted-vs-actual tracking.

Generated with reportlab from project figures. Metrics recovered from House_Prices_Report.docx and verified against house_prices.py. Reproduce the pipeline with `python house_prices.py`.